

Synthèse Project flutter

Lien Github : https://github.com/TksDesign/flutter_project.git

I. Quiz app :

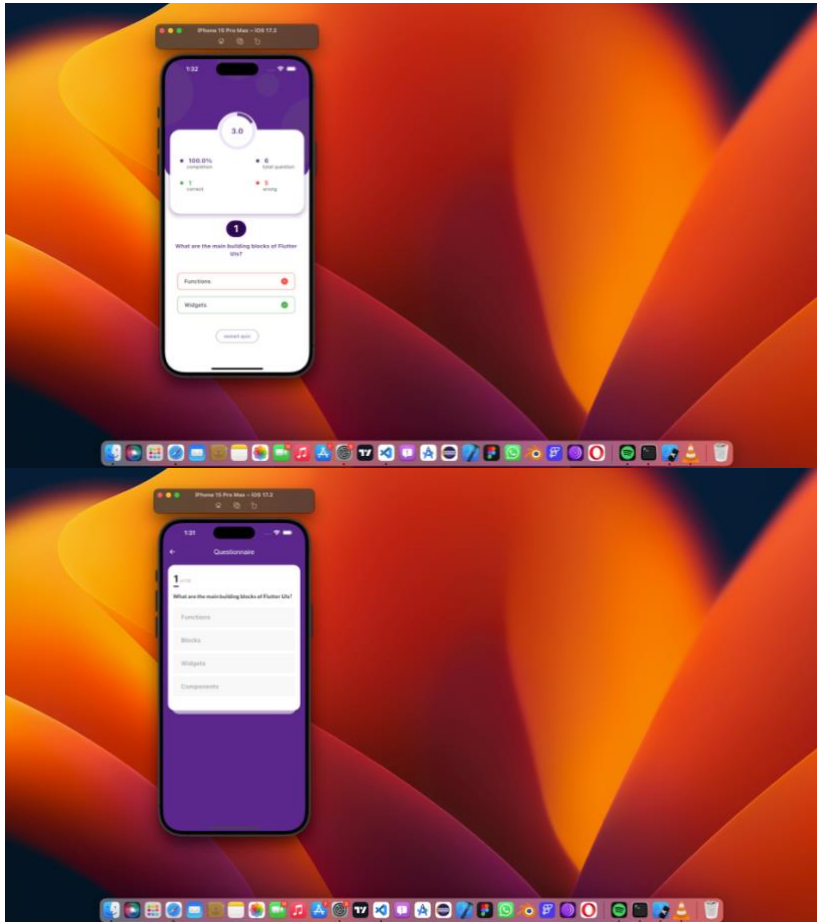
ici il était question d'apprendre la gestion des pages, de la réutilisation d'un composant et de la gestion des class
l'application en elle permet de soumettre un ensemble de question et de voir le résultat aux questions de l'utilisateur

point important :

- L'utilisation type « **Map** » et sa différentiation par rapport au type « **map** »
- utilisation de la fonction `shuffle()` pour pouvoir recopier une liste où l'ordre des questions changent
- utilisation des modes de debugging (profil-mode, debug-mode)

```
- class QuizQuestion {  
-   const QuizQuestion(this.text, this.answer);  
-   final String text;  
-   final List<String> answer;  
-  
-   List<String> getshuffledAnswers() {  
-       final shuffledList = List.of(answer);  
-       shuffledList.shuffle();  
-       return shuffledList;  
-   }  
- }  
-  
-
```

- Manipulation de package GoogleFont



II. Expense Tracker APP

Ici il était question d'apprendre plus en profondeur la gestion des datas (add, delete), des classes dans une application et du responsive en fonction de la taille d'écran.

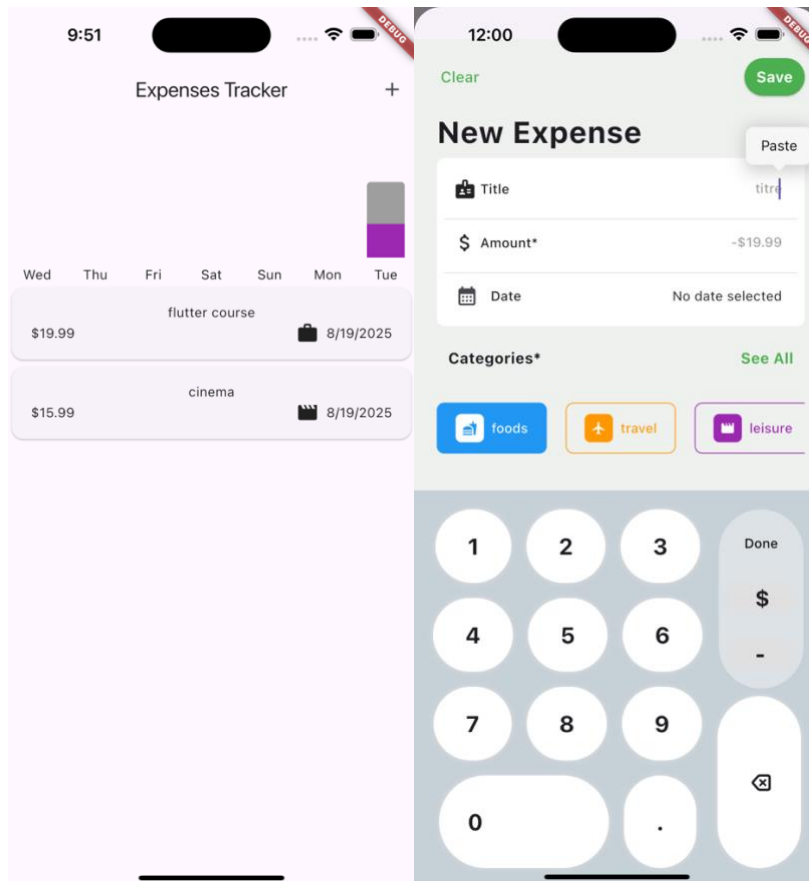
L'application en soit consiste à faire un suivi des dépenses quotidiennes et était faite en deux étapes : tout d'abord l'application fonctionnelle puis le responsive en fonction de la taille d'écran et du type d'émulateur Android/iOS.

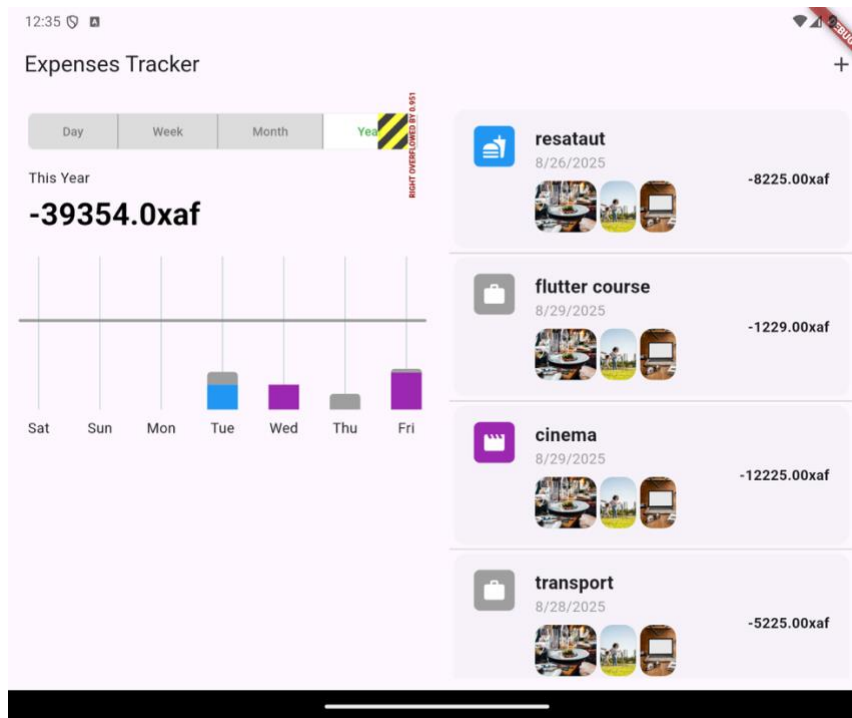
point important :

- Utilisation du widget **dismissible** pour supprimer de façon fluide d'un composant
- L'utilisation de l'attribut **Key** pour l'identification unique d'un élément
- Utilisation du widget **textfield** pour la saisie

- Utilisation de la fonction **Where** pour la gestion des map ou tableau de façon plus spécifique
- Utilisation de package **uuid** pour la gestion des id
- Utilisation du MediaQuery pour la gestion des taille d'écran

- `final width = MediaQuery.of(context).size.width;`





III. Meals App

Ici on a d'abord commence par comprendre le concept d'arbre de widget et de rendu de composant c'est-à-dire la relation entre :

Widget Tree – Element tree – Render Tree

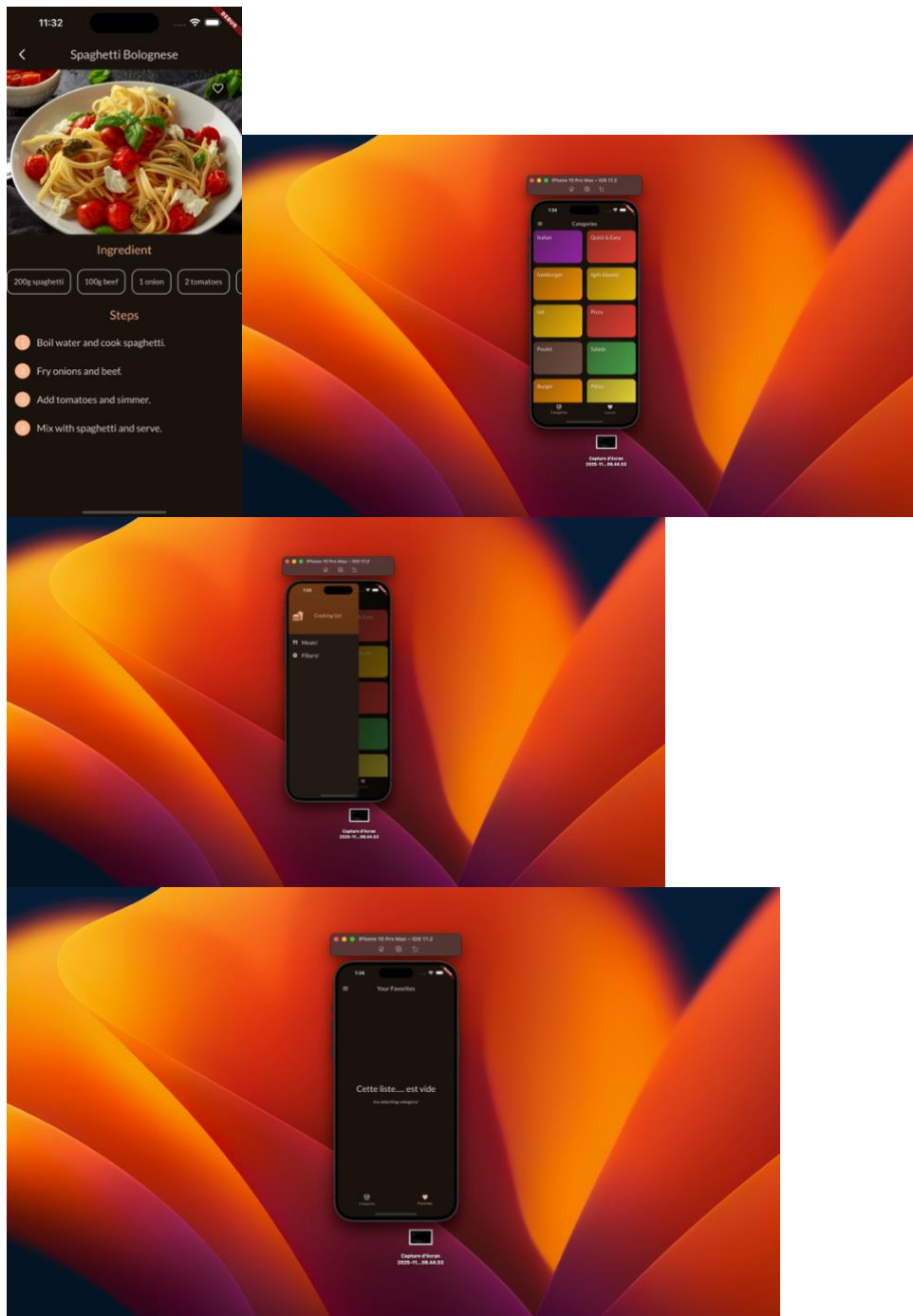
Puis on a continué par les gestions des donnes dans une application tout abords par la transmission des données d'un widget parent vers un widget enfant et vis vers ca et aussi de la transmission des données d'un petit fils vers le un parent ; ensuite on a vu comment simplifie la transmission de donnée pour enfin voir les gestion des animation entre différents page

et des types d'animations

l'application en elle-même est une application qui permet d'afficher des listes de repas en fonction du type et de la composition et de pouvoir filtrer les éléments qui seront afficher en fonction de leur composition et de l'ajout au favorite

point important :

- Utilisation du `Navigator.pop` et `Navigator.push` pour la transmission de donne
- Utilisation du gestionnaire d'états `Riverprod` pour la simplification lors de la transmission des donnees entres composant
- Utilisation du package `transparent_image` pour la gestion du chargement d'image
- Utilisation du `ThemeData` pour la gestion du theme general de l'application
- Utilisation du `BottomNaviagationBar` et du `DrawerHeader` pour la navigation entre page



IV. Shopping list

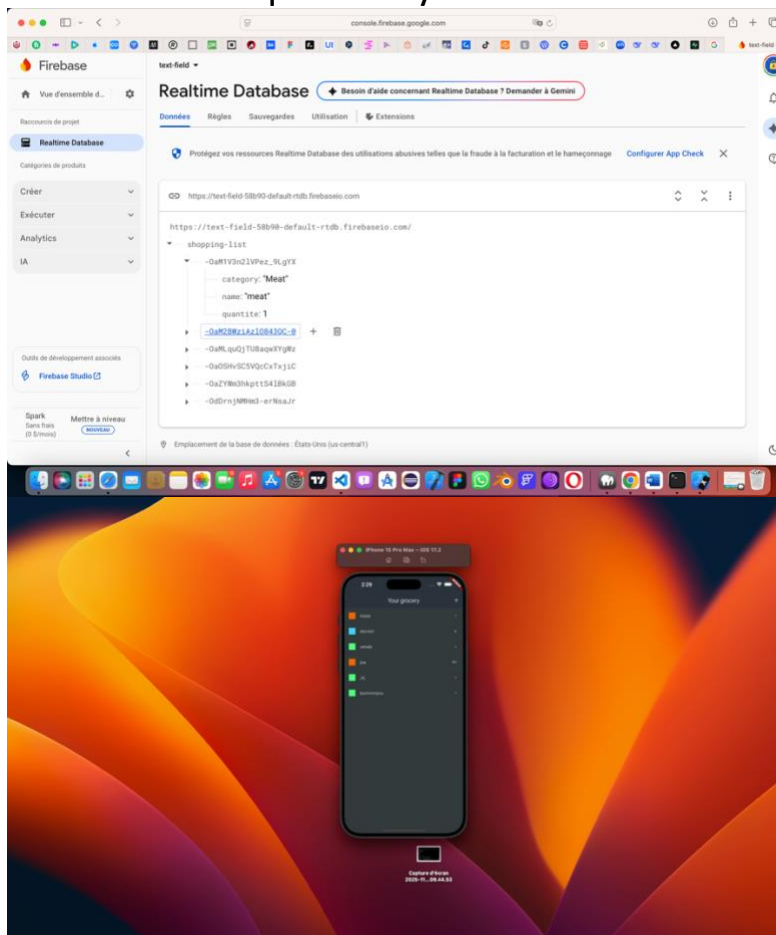
ici on a appris dans un premier temps a utiliser plus en profondeur les inputs avec un autre widget et a gérer encore l'affichages des données en fonction des données internes et

dans un deuxième temps la connexion au a Firebase pour l'envoi des données

L'application en elle-même est une sorte d'application de shopping où tu ajoutes des produits, affiche et supprime

point important :

- Utilisation du package **http** pour la gestion des requêtes https vers le backend
- Manipulation de **Realtime Database** de Firebase pour la gestion de la base de données grâce au NoSQL
- Gestion des requêtes asynchrone et





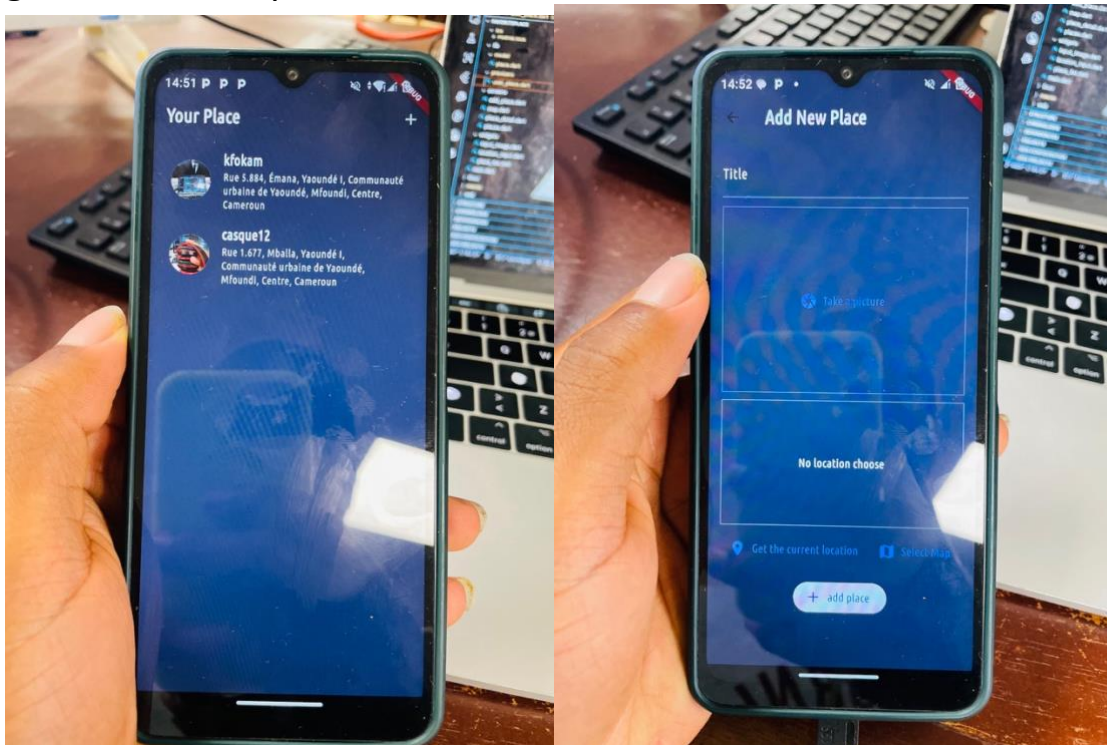
V. Favorite Place

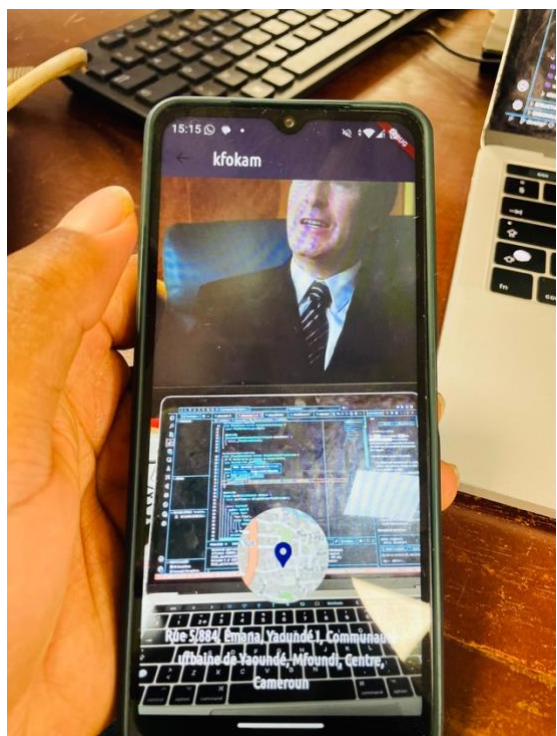
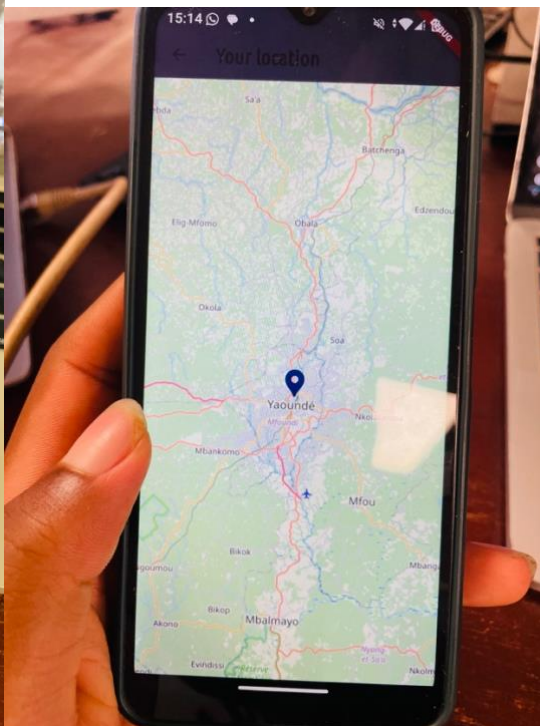
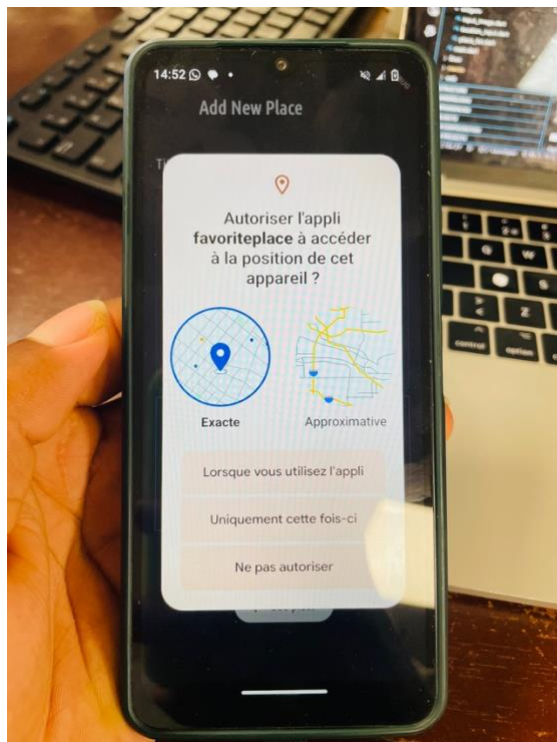
Ici on d'abord appris à utiliser les outils interne comment l'appareil photo la localisation ; pour ensuite utiliser l'api de google-map pour récupérer les localisations , afficher une carte en fonction de celle-ci et de pouvoir changer cette localisation sur la carte
puis dans second temps on appris à gérer le local Storage pour la gestion interne des données dans l'application
l'application en elle-même est une appli qui permet de garder en mémoire les lieux les plus aimés pour pouvoir les revisiter en cas d'oubli

point important :

- Utilisation du package `flutter_riverpod` pour la gestion simplifiée de l'état
- Utilisation du package `location` pour récupérer la localisation

- Utilisation du package **sqlite** pour la base de donne interne d'une application
- Utilisation du type **File** pour la gestion des images
- Utilisation du package **Path** pour la gestion des routes d'images
- Utilisation du type **Future** et du widget **FutureBuilder** qui permettent la gestion et le rendu des évènements future dit asynchrone
- Création de la base de donnee interne d'une application grâce aux déclarations comme **getDatabasesPath()** pour la création et la gestion des scripts de création de base données





VI. Chat app

Ici on appris tout d'abord une plus grand gestion de l'état reactif d'une application grâce a l'utilisation des websocket meme si celui-ci état simplifier par la backend SuperBase avec qui on a aussi appris l'authentification utilisateur , le stockage d'images/video , on a appris l'utilisation firebase database qui utilise ici postgreSql pour ca base données et on a join a ca les notification push de firebase cloud meseging et pour finir cette fois si l'utilisation du widget **StreamBuilder** pour la communication réactive entre super base et l'application

l'application en elle-même est une application de chat de groupe

